

L'ABC de l'administration système Unix

A. Michelizza

3 septembre 2010

Table des matières

1	Administrer un système Unix	1
2	Evaluer l'état global de la machine	2
2.1	Inspecter les <i>logs</i>	3
2.1.1	Les logs, c'est quoi?	3
2.1.2	Les logs, c'est où?	3
2.1.3	Chercher dans les fichiers de logs	4
2.2	Mesurer l'espace disque disponible	5
2.3	Mesurer la consommation de la mémoire	5
2.4	Mesurer la consommation de la CPU	8
2.5	Mesurer la consommation d'IO	9
3	Que faire si...?	11
3.1	Que faire si une partition est pleine	11
3.2	Que faire si une partition est <i>presque</i> pleine	11
3.3	Que faire si il n'y a plus assez de mémoire?	12
3.4	Que faire si il n'y a plus assez de CPU?	12
3.5	Repérer les processus qui consomment trop de ressources	13
4	Redémarrer un processus	13
4.1	Relancer les services avec les scripts d' <i>init</i>	13
4.2	Tuer directement un processus	14
5	Redémarrer le système	15
6	Diagnostiquer un problème réseau	16
6.1	Tester un service TCP	16
6.2	Tester la connection IP vers une machine distante	17
6.3	Tester la résolution de nom	18
6.4	Connaître son adresse IP et analyser la configuration réseau	19

1 Administrer un système Unix

Chargé du bon fonctionnement des systèmes, des applications, mais aussi parfois du réseau et des bases de données, les attributions de l'administrateur système recouvrent des tâches très variées. Il devra être capable de procéder

à l'installation des systèmes et des applications, leur paramétrage, leur maintenance, leurs mise-à-jour, leur supervision, les sauvegardes, le support... une importante liste de tâches qui pourtant est loin d'être exhaustive!

Administrer un système Unix demande donc des connaissances et des compétences étendues. Outre une bonne connaissance du fonctionnement global du système, allant des couches les plus basses du noyau aux couches applicatives, il faut en maîtriser les outils et avoir une bonne connaissance du réseau et du protocole TCP/IP. Mais atteindre ce niveau de compétences ne se fait pas en un jour ni tout seul : il faut de l'étude et de la pratique!

Ce cours aborde les concepts et les outils que vous devez absolument connaître pour résoudre les problèmes les plus courants. On peut en fait comparer l'administrateur système à un médecin. Les médecins ont tous fait des années d'étude et ils possèdent de ce fait des connaissances et un savoir faire leur permettant de diagnostiquer et de soigner des centaines de maladies différentes. Cependant, au quotidien, ils traitent surtout des rhumes, des bronchites et des maux de ventre! Sous Unix, c'est pareil : les problèmes qui surviennent sont souvent les mêmes. En premier lieu, nous allons donc voir comment établir le diagnostic de base, puis nous verrons quoi faire dans les quelques cas les plus simples.

Si ce support ne vous suffit pas et que vous souhaitez développer davantage vos connaissances en administration système sous Unix, la première source d'information est Internet. La plupart des pages sont en anglais et il vous sera difficile de faire l'impasse sur cette langue à partir d'un certain niveau de compétence et d'expertise. Sinon, je recommande la lecture de deux ouvrages de référence, l'un en administration système, l'autre concernant le protocole TCP/IP :

- *Evi Nemeth, Garth Snyder, Trent R. Hein, Ben Whaley. UNIX and Linux System Administration Handbook. Prentice Hall, 4th edition, 2010.*

Pour tout savoir, tout comprendre, tout maîtriser! Cette somme de 1344 pages, qui condense des années d'expérience, est la référence absolue.

- *Douglas Comer. TCP/IP. Architecture, protocoles et applications. Pearson Education, 5e édition, 2009.*

Un excellent ouvrage, très accessible au débutant, qui permet d'aller assez loin dans la compréhension des différents protocoles. Un autre ouvrage de référence.

2 Evaluer l'état global de la machine

Vous avez remarqué que le médecin commence toujours sa consultation en demandant au malade ce qui ne va pas, ce qu'il ressent. L'administrateur système va procéder de la même façon : il va faire "parler" le système grâce aux fichiers de *logs*. Le médecin a également à sa disposition un ensemble d'instruments et de techniques pour évaluer rapidement notre état. Prise de tension, auscultation et prise du pouls fournissent rapidement de bonnes indications quant à notre santé globale. L'administrateur système dispose lui aussi d'outils simples permettant d'avoir rapidement un aperçu général de l'état de la machine! Nous allons détailler ces outils qui permettent de mesurer les variables de base du système :

- la consommation de l'espace disque
- la consommation de mémoire
- l'activité du processeur, la consommation du CPU

- le nombre d'entrées / sorties (IO)

2.1 Inspecter les *logs*

2.1.1 Les logs, c'est quoi ?

Vous êtes appelé en urgence pour un problème sur un serveur. Quelque soit la nature du problème, la solution se trouve très certainement dans l'un des journaux de bord (*log* en anglais) qui consigne minutieusement toute son activité!

Sur un système Unix, quasiment chaque processus écrit dans les logs pour informer l'opérateur de son activité. En général, un processus ne va reporter que des informations méritant d'être notifiées, utiles à la résolution de problèmes ou à la compréhension de ce qui se passe. Problèmes critiques, échecs d'envoi ou de réception, informations de débogage... le niveau de granularité de ces informations est presque toujours paramétrable au niveau de l'application elle-même.

2.1.2 Les logs, c'est où ?

Les *log* sont toujours dans `/var/log`. Cet endroit est standard et il ne varie en principe pas d'un système Unix à l'autre. La mise à jour de ces logs est effectué par deux mécanismes différents :

- Dans le premier cas, les processus écrivent directement leur logs dans un fichier ou un sous-répertoire qui leur est propre. Par exemple, le démon web *httpd* va notifier ses erreurs dans `/var/log/httpd/error.log` et le démon mail *sendmail* va écrire dans le fichier `/var/log/maillog`.
- Dans le second cas, les processus transmettent leur logs à un programme dédié à la collecte de ces informations : le démon **syslogd**.

Ce démon reçoit les informations de journalisation des processus, il les trie, puis il les consigne dans un fichier de `/var/log` en ajoutant certaines informations utiles, telles que la date, le nom de l'application ou son *pid* :

```
$ sudo tail -5 /var/log/syslog
Jul 30 12:39:00 fnord slapd[1118]: connection_read(25): no connection!
Jul 30 12:39:00 fnord slapd[1118]: connection_read(25): no connection!
Jul 30 12:39:01 fnord pulseaudio[1953]: ratelimit.c: 64 events suppressed
Jul 30 12:39:05 fnord CRON[5848]: pam_unix: session opened for user root by (uid=0)
Jul 30 12:39:05 fnord slapd[1118]: connection_read(25): no connection!
```

Tous les logs ne vont pas être écrits dans le même fichier. Le démon **syslogd** utilise généralement plusieurs fichiers selon le type d'informations à retranscrire :

```
$ cd /var/log
$ ls -ltr | tail -5
-rw-r----- 1 syslog      adm      105272 2010-07-30 12:35 auth.log
-rw-rw-r-- 1 root         utmp      309504 2010-07-30 12:36 wtmp
-rw-r----- 1 syslog      adm       20061 2010-07-30 12:37 user.log
-rw-r----- 1 syslog      adm       11164 2010-07-30 12:37 syslog
-rw-r----- 1 syslog      adm       196095 2010-07-30 12:37 messages
```

Mais en général, les fichiers `/var/log/messages` et `/var/log/syslog` centralisent l'ensemble des informations.

2.1.3 Chercher dans les fichiers de logs

Comment trouver parmi l'ensemble des fichiers de journalisation les informations susceptibles de nous éclairer? Non seulement les fichiers de logs sont nombreux, mais en plus, ils peuvent rapidement devenir très volumineux si l'activité système est intense ou que leur réglage fait qu'un (trop?) grand nombre d'informations sont consignées. Pour vous retrouver dans cette jungle de données, il va falloir vous munir de votre coupe-coupe favori : les commandes `ls`, `tail` et `grep`!

Repérer les fichiers les plus récemment mis à jour

Les informations les plus récentes sont les plus susceptibles de nous apporter des éléments intéressants concernant l'état de la machine. Jusque là, ça semble assez logique! Les commandes ci-dessous permettent de repérer les fichiers les plus récemment mis à jour :

```
$ cd /var/log
$ ls -ltr | tail -5
```

Observer un ou plusieurs fichiers

Pour voir ce que `syslogd` est en train d'écrire au moment où vous êtes connecté à la machine, il faut utiliser la commande `tail` avec l'option `-f`. Par exemple, pour suivre l'activité du fichier `messages` :

```
$ tail -f messages
```

Comme pour la quasi totalité des commandes sous Unix, la commande `tail` permet d'observer plusieurs fichiers en même temps. Par exemple :

```
$ tail -f messages syslog cron
```

Récupérer une information spécifique

Supposons que vous soyez sur la piste d'un problème de mail concernant l'utilisateur `toto@mondomaine.fr` qui ne parvient pas à émettre de message. Vous avez remarqué que le fichier `/var/log/maillog` contient les informations relatives au mail. C'est donc le bon endroit où chercher. Le problème est que ce fichier se remplit très vite et il est impossible de visualiser correctement les nouvelles informations qui défilent trop rapidement! Heureusement, la commande `grep` est votre amie :

```
$ tail -f maillog | grep toto@mondomaine.fr
```

Et si cela ne suffit pas, n'oubliez pas la commande `more` (ou `less` si vous préférez) :

```
$ tail -f maillog | grep toto@mondomaine.fr | more
```

Inspecter les logs du noyau

Le noyau aussi, à l'instar des applications de plus haut niveau, produit des logs qui peuvent être intéressants :

```
$ dmesg | more
```

2.2 Mesurer l'espace disque disponible

Un disque plein ou un système de fichier saturé et l'activité du serveur peut se trouver entièrement paralysée ! La commande `df` permet de mesurer pour chaque volume l'espace disque disponible :

```
$ df -h
Filesystem            1K-blocks      Used Available Use% Mounted on
/dev/mapper/VolGroup00-LogVol100
                        23551868    1488216   20847960    7% /
/dev/sda1              101086      12623     83244    14% /boot
tmpfs                  12337736         0   12337736    0% /dev/shm
```

Dans le cas ci-dessus, les disques sont très peu utilisés. Le volume commençant à la racine (/) est plein à 7% seulement et le volume attaché au répertoire `/boot` est utilisé à 14%. La dernière ligne de cette entrée, concernant `tmpfs`, est un peu particulière et ne sera pas détaillée ici (on observe toutefois que ce `tmpfs` est inutilisé).

Un petit conseil pour davantage de lisibilité : la commande `df` des systèmes Linux supporte l'option `-h` pour afficher les volumétries en mega ou en giga plutôt qu'en kilo (octets). Dans tous les cas, et il faut que ça devienne pour vous une habitude sous Unix : faire appel à la commande `man` pour voir toutes les options disponibles.

Une autre option permet de voir le nombre de fichiers que l'on peut encore créer sur un volume. Il faut en effet savoir qu'il n'est pas possible de créer une infinité de fichiers dans un système de fichiers car celui-ci est limité :

```
$ df -i
Filesystem            Inodes    IUsed   IFree IUse% Mounted on
/dev/mapper/VolGroup00-LogVol100
                        6082944   40826  6042118    1% /
/dev/sda1              26104      35   26069    1% /boot
tmpfs                  3084434      1  3084433    1% /dev/shm
```

Là encore, la lecture des pourcentages exprimés indique une très faible utilisation de l'espace disque.

2.3 Mesurer la consommation de la mémoire

Un problème fréquent provient du fait que la mémoire du système est à la libre disposition de l'ensemble des processus. Or si un processus décide de consommer à lui seul l'ensemble de cette mémoire, l'impact sur l'ensemble du système, sans même parler des autres processus, est catastrophique ! L'ensemble du système se met alors à fonctionner au ralenti jusqu'à la paralysie complète de la machine. Le manque de mémoire est donc un problème majeur à traiter sans délai.

La commande free

Pour mesurer la consommation de la mémoire, il existe plusieurs commandes, chacune avec sa spécificité. La commande **free** fournit des informations détaillées sur la façon dont la mémoire est consommée :

```
$ free -m
```

	total	used	free	shared	buffers	cached
Mem:	4054	3893	161	0	179	3322
-/+ buffers/cache:		390	3663			
Swap:	4095	0	4095			

La première ligne indique la consommation de la mémoire physique, c'est à dire celle fournie par les barrettes de mémoire :

- **total** indique la quantité de mémoire physique totale.
- **used** indique la quantité de mémoire physique consommée par le système dans son ensemble.
- **free** indique la quantité de mémoire physique inutilisée.
- **shared** est obsolete et doit être ignoré.
- **buffers** et **cached** renseignent sur la mémoire utilisée pour (essentiellement) optimiser les performances du système. Cette mémoire peut être rendue disponible à tout moment.

Dans l'exemple ci-dessus, le système dispose en tout de 4054 Mo de mémoire. Sur ce total, 3893 Mo sont utilisés et 161 Mo sont inutilisés. A première vue, toute la mémoire est consommée et la machine est donc dans un état critique. Pourtant, ça n'est pas du tout le cas. Attention à ne pas confondre mémoire inutilisée et mémoire *disponible* ! Le système consomme un maximum de mémoire pour optimiser au mieux ses performances via l'utilisation de caches, et il n'y a donc que très peu de mémoire inutilisée (colonne **free**). Mais la mémoire utilisée par les caches (colonnes **buffers** et **cached**) peut être libérée à tout moment.

La deuxième ligne indique la quantité de mémoire réellement consommée par les applications (colonne **used**) et la mémoire réellement disponible (colonne **free**). En cessant d'utiliser les caches, le système ne consommerait donc plus que 390 Mo de mémoire alors que 3663 Mo seraient disponibles. Finalement, ce serveur a encore beaucoup de mémoire disponible en réserve !

A quoi servent les *buffers* / *cached* ?

Lire un fichier, le modifier, écrire dedans... toutes ces actions ont des répercussions importantes sur les performances du système car les accès disques sont très lents comparés aux accès en mémoire. Pour minimiser les accès disques en lecture et en écriture, le noyau conserve en mémoire aussi longtemps que possible les informations lues ou écrites. Travailler sur une copie en mémoire est alors très rapide. Mais attention au *crash* ! Pour ne pas perdre l'ensemble de ces données en cas de panne du système, elles sont écrites sur le disque à intervalle régulier (généralement 5 secondes).

La dernière ligne indique la consommation de mémoire *swap*. Le *swap* est une partition particulière du disque dur utilisée par le noyau pour étendre la mémoire physique. Ce besoin découle de la différence entre le prix de la mémoire qui est beaucoup plus cher que celui des disques. Pourquoi dans ce cas ne pas se

servir du disque comme si c'était de la mémoire physique? C'est exactement le rôle du *swap* qui permet d'étendre considérablement la mémoire utilisable par le système. En revanche les accès disques étant beaucoup plus lents, cet espace ne sera utilisé qu'en cas d'extrême nécessité, quand la mémoire physique est épuisée.

Quand le système se met à utiliser son swap (on dit alors qu'il "swappe"), les conséquences sur les performances globales du système sont très visibles car celui-ci est alors considérablement ralenti. Le swap doit donc essentiellement être vu comme une mémoire de secours, à utiliser le moins possible. Dans l'exemple ci-dessus, la colonne **used** avec une valeur de 0 indique que le swap n'est pas utilisé, ce qui est plutôt rassurant pour cette machine.

Notez que certains noyau utilisent une petite portion du swap pour y entreposer de façon temporaire des informations. Cela n'a rien d'alarmant tant que cette consommation est modérée et qu'il reste suffisamment de mémoire physique de libre.

La commande `vmstat`

La commande `vmstat` permet de suivre en temps réel l'utilisation de la mémoire :

```
$ vmstat 1
procs -----memory----- --swap-- ----io---- --system-- -----cpu-----
 r  b swpd  free  buff  cache   si   so    bi    bo    in   cs us sy id wa st
 0  0   68 171036 184020 3399464    0    0    30     9     0    1  2  1 86 10  0
 0  1   68 168184 184020 3399464    0    0     0  1500 1908 1013  2  1 75 22  0
 0  0   68 170416 184028 3399492    0    0     0  1760 1775 1327  2  2 79 18  0
 0  0   68 170416 184028 3399492    0    0     0   464 1315  832  2  1 92  5  0
 0  1   68 168432 184032 3399488    0    0     4   636 1521 1426  3  1 91  4  0
 0  1   68 170788 184032 3399488    0    0     4  1132 1384  770  1  1 82 16  0
```

Dans l'exemple ci-dessus, une nouvelle ligne de rapport est affichée par `vmstat` toutes les secondes. La première ligne de rapport ne fournit que des informations statistiques et doit être ignorée. Les lignes suivantes fournissent en revanche de très nombreuses informations relatives notamment à la consommation de CPU et de mémoire. Nous n'examinerons ici que les informations relatives à cette dernière.

Les colonnes **free**, **buff** et **cache** sont très importantes car ce sont elles qui nous renseignent sur la mémoire physique encore disponible (en kilo-octets). La colonne **free** indique la quantité de mémoire inutilisée, les colonnes **buff** et **cache** indiquent les quantités utilisées par les buffers et le cache. Comme les buffers et le cache sont libérables à tout moment, la mémoire réellement disponible correspond à la somme de ces trois indicateurs.

La colonne **swpd** montre la quantité de swap utilisée. Dans le cas présent, les 68 Ko utilisés n'ont rien d'alarmant car la mémoire physique disponible est encore très importante et cette consommation n'est donc pas liée à une quelconque pénurie. Elle sert juste au noyau à entreposer temporairement quelques informations.

Les deux colonnes suivantes, **si** et **so**, nous fournissent de très précieux renseignements quant à la santé du système. Nous avons vu plus haut qu'en cas de pénurie de mémoire physique, le noyau pioche dans le swap. En fait, le swap n'est pas utilisé exactement comme de la mémoire physique mais plutôt comme

un entrepôt intermédiaire. Quand le noyau souhaite utiliser une information en swap, il doit d'abord la transférer en RAM. Ce type de transferts est signalé par la colonne **si** (*swap in memory*). Mais... n'avons nous pas dit qu'il n'y avait plus assez de RAM libre? Exactement, et c'est pourquoi le noyau doit d'abord faire un peu de place en transférant des informations de la RAM *vers* le swap. Ces transferts sont signalés par la colonne **so** (*swap out of memory*). La présence de ces transferts est un indicateur fort de la pénurie de mémoire physique. Un système qui utilise son swap passe son temps à faire de coûteux allers et retours entre le disque et la RAM. C'est donc un système aux performances très dégradées.

2.4 Mesurer la consommation de la CPU

Un système Unix est multi-tâches, cela signifie que plusieurs programmes peuvent s'exécuter en même temps sur la même machine. Cela peut sembler idyllique, et ça l'est en grande partie, mais la réalité est malheureusement parfois beaucoup moins rieuse!

En fait, les processus ne s'exécutent pas vraiment en même temps. Le noyau s'arrange pour partager le temps CPU entre les différents processus mais à un instant donné, seul l'un d'eux s'exécute pendant un tout petit laps de temps. Passé cet intervalle, le noyau va mettre interrompre le processus en cours pour en exécuter un autre, et ainsi de suite. Cela signifie que sur une machine très chargée (avec beaucoup de processus), un processus va mettre plus de temps à s'exécuter car il devra attendre plus longtemps que le noyau lui donne la main.

Qu'est-ce qu'un processus?

Un processus est tout simplement un programme en train de s'exécuter. Un *démon* est un type un peu particulier de processus qui tourne en arrière plan, hors du contrôle direct d'un utilisateur. Par exemple, la séquence **^C** (*controle-C*) ne permettra pas de le stopper. En général, un démon est lancé au démarrage de la machine et il sert des requêtes réseau (mais pas seulement), comme par exemple le démon **apache** qui répond aux requêtes HTTP pour fournir des pages web.

La commande **vmstat**

La commande **vmstat**, déjà vue précédemment pour analyser la consommation mémoire, permet de suivre en temps réel l'utilisation de la CPU :

```
$ vmstat 1
procs -----memory----- --swap--  ----io----  --system--  -----cpu-----
 r  b swpd  free  buff  cache  si  so    bi    bo    in   cs   us sy id wa st
0  1   68 159976 217660 3380524  0  0    27    27    1    0    2  1 86 10  0
0  1   68 161216 217660 3380612  0  0    0 4000 2266 1091  4  1 71 24  0
0  1   68 161092 217660 3380612  0  0    0 3164 2346 1350  3  1 71 25  0
0  1   68 160596 217672 3380856  0  0    8 1976 1698 2175  5  2 69 23  0
0  1   68 160844 217676 3380852  0  0    0 1412 1572 1865  4  1 74 20  0
0  0   68 164192 217680 3380840  0  0    0  840 1383 1440  4  1 84 11  0
```

La colonne **r** indique le nombre de processus *attente d'exécution*. Cette appellation est un peu trompeuse car elle désigne en fait les processus prêts à s'exécuter et qui attendent que le CPU se libère mais aussi ceux qui sont en

cours d'exécution (qui utilisent le processeur). Si le nombre de processus en attente d'exécution est constamment supérieur au nombre de processeurs de la machine, cela signifie que le système est chargé. Est-ce que c'est critique ? Pas forcément ! Cela signifie juste que les processus doivent attendre leur tour pour s'exécuter. Savoir si cette charge est critique ou non va dépendre essentiellement de paramètres externes, tels que la qualité de service ou les délais de réponse souhaités.

Un deuxième indicateur de charge est fourni par la colonne **us**. Elle indique, en pourcentage, le temps passé par le système à exécuter des programmes applicatifs.

La colonne **sy** indique, en pourcentage, le temps passé par le système à exécuter des routines du noyau. Normalement, ces routines sont très rapides. Une valeur élevée peut donc être le signe d'un problème et demande des investigations supplémentaires. En général, cette valeur est plus faible que celle indiquée par la colonne **us**.

L'utilisation du processeur correspond donc à la somme des colonnes **us** et **sy**. Si cette valeur est constamment proche de 100, cela signifie que le processeur est très utilisé. Si en plus il y a de nombreux processus en attente, alors peut-être faudra-t-il revoir la politique de répartition de charge ? Mais là encore, c'est très relatif.

La colonne **id** indique la proportion de temps durant lequel le CPU est inutilisé. Si il y a un seul indicateur de charge à retenir, c'est peut-être celui-ci !

Notez que les trois indicateurs précédents (**us**, **sy** et **id**) sont établis à partir d'une moyenne de l'ensemble des processeurs. Par exemple, sur une machine quadri-processeurs, si l'un de processeur est utilisé à 100%, le pourcentage d'utilisation indiqué par **vmstat** sera de 25%. Pour connaître exactement la consommation par processeur, il faut utiliser la commande **mpstat**.

Les colonnes **in** et **cs** portent sur l'activité du noyau. Ces valeurs sont difficilement exploitables, on les ignore !

La commande uptime

La commande **uptime** indique la charge moyenne sur respectivement 1, 5 et 15 minutes. Par charge moyenne, on entend le nombre moyen de processus en attente d'exécution ou en sommeil :

```
$ uptime
10:11:56 up 20 days, 22:20, 1 user, load average: 0.67, 0.58, 0.46
```

Dans l'exemple ci-dessus, cette charge est de respectivement 0.67, 0.58 et 0.46. La signification de ces indicateurs n'est pas directement exploitable, même en connaissant le nombre de processeurs. Est-ce pour autant qu'ils sont inutiles ? Non, ils sont en fait très utiles quand ils sont intégrés à un outil de supervision qui va les grapher en temps réel et enregistrer leur historique. Les variations de charge constitueront alors des indicateurs précieux de l'activité de la machine.

2.5 Mesurer la consommation d'IO

Les problèmes liés aux entrées/sorties (*Input/Output* en anglais, ou *IO*) ne sont pas aussi courant que ceux liés à l'espace disque ou à la consommation

mémoire. Il faut toutefois être capable de les détecter ! Le nombre d'IO peut en effet impacter directement l'état global du système. Si par exemple trop de processus essayent en même temps de lire et d'écrire sur le même disque, l'engorgement lié à ce trop grand nombre de requêtes va pénaliser l'ensemble des processus concernés.

La commande `vmstat`

La commande `vmstat` permet de suivre en temps réel les impacts liés aux IO :

```
$ vmstat 1
procs -----memory----- --swap-- ----io---- --system-- -----cpu-----
r  b swpd  free  buff  cache  si  so  bi   bo   in   cs   us sy id wa st
0  1   68 159976 217660 3380524 0  0  27   27   1   0   2  1 86 10  0
0  1   68 161216 217660 3380612 0  0   0 4000 2266 1091  4  1 71 24  0
0  1   68 161092 217660 3380612 0  0   0 3164 2346 1350  3  1 71 25  0
0  1   68 160596 217672 3380856 0  0   8 1976 1698 2175  5  2 69 23  0
0  1   68 160844 217676 3380852 0  0   0 1412 1572 1865  4  1 74 20  0
0  0   68 164192 217680 3380840 0  0   0  840 1383 1440  4  1 84 11  0
```

En premier lieu, la colonne `b` indique le nombre de processus en *sommeil ininterruptible*. Pour l'essentiel ce sont des processus qui attendent qu'une entrée/sortie vers un périphérique soit réalisée. Dans cette attente, ils sont bloqués par le système (mis en *sommeil*).

La colonne `wa` indique le temps passé par le système à attendre que des lectures ou des écritures soient effectuées. Quand un processus essaye par exemple de lire des données sur un disque, le noyau le met en sommeil en attendant que celles-ci soient disponibles. Normalement, ce temps de sommeil est très court, voire nul grâce à l'utilisation des caches. Il s'agit donc d'un indicateur essentiel pour estimer si il y a des engorgements et le nombre de processus en sommeil ininterruptible est à rapprocher de cette valeur. Des valeurs de `b` et de `wa` constamment non nulles sont le signe d'un engorgement au niveau des IO.

Les colonnes `bi` et `bo` renseignent sur l'activité disque, respectivement les lectures et les écritures. En général, on se sert assez peu de cet indicateur car il n'est pas aussi précis que `iostat`.

Qu'est-ce qu'un processus *bloqué en sommeil non interruptible* ?

Quand un processus tente de lire un fichier sur une unité de disque, il ne peut le faire directement car il n'a ni la permission, ni la connaissance, de comment interagir directement avec le matériel. Le processus est obligé de faire appel à une fonction spécialisée du noyau : un *appel système*. Dans le cas de certains appels systèmes, le noyau met le processus en *sommeil ininterruptible* jusqu'à la fin de l'appel, ce qui ne pose aucun problème car les appels systèmes s'exécutent très rapidement. Mais dans certains cas, il peut arriver que l'appel se "bloque", souvent en raison d'un problème au niveau d'un périphérique ou d'un bug du noyau, et le processus ne se réveille jamais ! Les processus en sommeil ininterruptible peuvent être repérés avec la commande `ps` : la valeur de la colonne `STAT` est à D.

3 Que faire si... ?

3.1 Que faire si une partition est pleine

Si une partition est pleine, la solution est évidente... il faut faire de la place ! Pourtant, cela est beaucoup plus facile à dire qu'à faire.

Il faut commencer par détecter les fichiers qui consomment le plus de place dans la partition. Ça n'est pas si facile qu'il y paraît car un volume disque peut contenir des milliers de fichiers. En général, les plus gros fichiers sont directement sous `/var/log`, on peut donc commencer par chercher à cet endroit :

```
$ cd /var/log
$ ls -alrS
```

La commande `du` est aussi très pratique car elle permet de voir la taille des différents sous-répertoires d'une arborescence et de les classer selon leur volume :

```
$ cd /var/log
$ du -sk * | sort -n
```

Admettons que nous ayons trouvé sur notre système un très gros fichier : `/var/log/httpd/error.log`. La première chose à faire est de comprendre pourquoi ce fichier est si gros en scrutant ce qui le remplit grâce à la commande `tail -f`. Dans un deuxième temps seulement on pourra essayer de le déplacer vers une partition disposant de suffisamment de place grâce à la commande `mv`. C'est assez rare qu'on décide de supprimer un fichier, même si c'est une solution simple. Les informations contenues dedans peuvent s'avérer trop importantes, car directement liées à l'activité de l'entreprise ou soumises à des contraintes juridiques (pour un fournisseur d'accès, c'est par exemple le cas des journaux permettant de tracer l'activité de ses clients).

3.2 Que faire si une partition est *presque* pleine

A la différence d'une partition totalement pleine, une partition sur laquelle il reste de la place nous offre une option supplémentaire : compresser le fichier trop volumineux :

```
$ gzip big_file
```

La commande `gzip` supporte plusieurs options, et notamment une qui permet d'ajuster le taux de compression (de 1 à 9). Un taux de compression élevé permet d'obtenir un fichier plus petit, mais il implique des calculs plus longs et plus coûteux en terme de CPU. Si la machine est chargée, un taux de compression faible peut donc être souhaitable :

```
$ gzip -3 big_file
```

Pour vous donner une idée, un fichier texte de 100 Mo comprimé avec les différents taux possibles donne le tableau de résultats suivant :

Taux	Durée (s)	Taille (M)
1	1.136	9.2
2	1.156	8.8
3	1.204	8.7
4	1.508	8.1
5	1.600	7.8
6	1.976	7.4
7	2.224	7.3
8	3.776	7.1
9	4.544	7.0

On remarque que le taux de compression le plus faible (1) produit un résultat somme toute très acceptable en un temps 4 fois moins important qu'avec le taux de compression maximum. Pour information, le taux -6 est le taux de compression par défaut.

3.3 Que faire si il n'y a plus assez de mémoire ?

La pénurie de mémoire sur un serveur est rapidement dramatique en termes de performances. L'utilisation de l'espace de *swap* n'est qu'une solution de secours pour continuer d'assurer le fonctionnement du système et l'empêcher de *crasher* directement. Mais son utilisation est en soi un problème majeur. Autrement dit, votre serveur est en danger et il est temps d'agir !

Il y a en général deux raisons qui peuvent expliquer cette pénurie de mémoire :

- La quantité de mémoire du serveur a été sous-dimensionnée en vue de son utilisation courante.
- Un processus est victime d'un *memory leak*.

Le premier cas concerne une machine dont le fonctionnement ne comporte pas d'anomalie notable, elle manque juste de mémoire. Dans ce cas là, il est rare que le fonctionnement soit complètement ralenti, il faudra juste planifier une intervention afin de rajouter de la mémoire.

Le second problème concerne un processus qui se met à consommer de façon anormale toute la mémoire. Un *memory leak* (fuite de mémoire) se rapporte au fonctionnement d'un processus qui consomme de la mémoire sans jamais la rendre au système. Il s'agit en général d'un *bug* (une erreur de programmation) qui doit à terme être corrigé par une mise à jour du programme ou du package. Le graphe d'utilisation de la mémoire du serveur permet de mettre facilement en évidence l'existence d'un *memory leak*. Ces graphes ont un aspect en "dent de scie" avec une consommation mémoire qui augmente linéairement jusqu'à épuisement complet de celle-ci.

En cas de *memory leak*, il n'y a en général qu'une seule chose à faire : arrêter le processus fautif et le relancer. Toutefois, il s'agit juste d'une action à très court terme qui ne peut que temporairement soulager la machine.

3.4 Que faire si il n'y a plus assez de CPU ?

Contrairement à la pénurie de mémoire, le manque de CPU n'est en général pas trop pénalisant pour le serveur. Il faut cependant savoir qu'à l'exception des programmes utilisés pour faire des calculs intensifs ou des bases de données, l'essentiel des processus consomment assez peu de CPU. Par exemple, dans le

cas d'un serveur HTTP ou SMTP, l'essentiel de l'activité concerne les IO liés à des lectures/écritures sur le disque ou à l'activité réseau et la CPU consommée reste très faible. Dans ce contexte d'utilisation, la présence d'un processus qui exploite presque toute l'activité d'un processeur est une anomalie!

Les seuls remèdes pour réduire la charge sont d'ajouter un processeur ou de répartir l'exécution des processus sur plusieurs machine. Les commandes **ps** et **top** permettent de repérer les processus les plus gourmands en CPU.

Un processus qui utilise 25% de CPU... c'est grave ?

Tout dépend du nombre de processeurs! N'oubliez pas que sur un système à 4 processeurs, cela signifie que le processus utilise un processeur à lui tout seul.

3.5 Repérer les processus qui consomment trop de ressources

Les commandes **ps** et **top** permettent de voir par processus leur consommation en mémoire et en CPU.

La commande ps

Cette commande sert à lister les processus existant (elle dispose de très nombreuses options que nous ne détaillerons pas ici). Pour afficher pour chaque processus du système sa consommation mémoire et trier ce résultat selon cette consommation :

```
$ ps aux | sort -n -k 4
```

La commande top

Cette commande est pratique car elle permet de voir en temps réel l'évolution des ressources du système et elle permet également de trier les processus selon leur consommation. A ce titre, trois sous-commandes sont utiles :

- **M** pour trier les processus selon leur consommation de mémoire
- **P** pour trier les processus selon leur consommation de CPU
- **q** pour quitter

4 Redémarrer un processus

Pour arrêter, démarrer ou relancer un processus, il existe en général deux méthodes. La première est d'utiliser les scripts d'arrêt/relance spécialement prévus à cet effet. La seconde est d'agir directement sur le processus par le biais d'un *signal*.

4.1 Relancer les services avec les scripts d'*init*

La plupart des processus sur lesquels l'administrateur système doit agir sont des démons lancés au démarrage de la machine. Or ces démons sont lancés par le biais de scripts d'*init*. Ces scripts, présent dans `/etc/init.d`, sont exécutés au démarrage du système afin de l'initialiser et de démarrer les différents services. Ces scripts doivent être exécutés avec un argument de type **start** ou **stop**

qui indique si l'on souhaite démarrer ou arrêter le service. Par exemple, pour démarrer le démon de logs :

```
$ /etc/init.d/syslog start
```

Si on exécute le script sans rien lui indiquer, il affiche l'ensemble des arguments qu'il peut prendre :

```
$ /etc/init.d/syslog
Syntaxe : /etc/init.d/syslog {start|stop|status|restart|condrestart}
```

Et pour arrêter le démon de logs :

```
$ /etc/init.d/syslog stop
```

On peut vérifier que le processus a bien été terminé grâce à la commande `ps` :

```
$ ps ax | grep syslog
$ ps ax | head -1
```

Mais les scripts d'*init* ne fonctionnent pas toujours. Dans ce cas, il faut agir directement sur les processus.

4.2 Tuer directement un processus

Sous Unix, on peut agir sur un processus en lui envoyant un *signal* grâce à la commande `kill` qui est décrite un peu plus loin.

Pour tuer un processus, la première chose à faire est de l'identifier précisément. Cela semble a priori simple, car un processus est identifiable par le nom du programme exécuté?... Et bien non ! Si un programme est lancé plusieurs fois, les différentes instances de celui-ci portent toutes le même nom, il faut donc utiliser un autre moyen. Chaque processus est distingué par son *pid* (pour *process identifier*), un numéro unique qui peut être retrouvé grâce aux commandes `ps` et `top`.

La commande `kill`

Cette commande permet d'agir sur un processus en lui envoyant un *signal*. Sous Unix, les signaux peuvent être vus comme des sonnettes qui permettent de prévenir les processus de quelque chose. La liste et les effets de ces signaux est plus ou moins standard, et consiste en général à terminer le processus (malgré quelques variations d'un Unix à l'autre). Il faut juste retenir que en général :

- Le signal 15 (appelé **TERM**) est envoyé pour terminer le processus proprement.
- Le signal 9 (**KILL**), c'est le bazooka, l'artillerie lourde qui permet de terminer le processus en "urgence". A n'utiliser qu'en dernier recours !
- Le signal 1 (**HUP**) est en général implémenté de façon à ce que le démon relise son fichier de configuration.

Concrètement, comment ça se passe ? Avant toute chose, il faut récupérer le *pid* du processus à terminer puis on lui envoie le signal **TERM** :

```
$ ps ax | grep slowprog
29167 pts/0    R+      0:15 slowprog 50
$ kill 29167
```

Si la précédente commande a échoué, on peut sortir la grosse artillerie et envoyer le signal KILL :

```
$ kill -9 29167
```

Cette dernière commande devrait réussir, à moins que le processus est en *sommeil ininterrompible*, auquel cas la seule solution est de *rebooter* le système.

5 Redémarrer le système

En général, on ne doit jamais rebooter un serveur sous Linux ou tout autre Unix sauf cas vraiment exceptionnel ! L'essentiel des problèmes sous Unix peuvent en effet être résolus sans avoir à tout redémarrer. La première question à se poser avant de rebooter un serveur est donc de savoir si vous avez vraiment besoin de le faire.

Une fois votre décision prise, n'oubliez pas de prévenir les gens autour de vous. Le métier d'administrateur système n'est pas que technique, c'est aussi un métier de communication ! Gardez à l'esprit que des services absolument vitaux aux yeux de certains de vos interlocuteurs vont être interrompus au moment du *reboot*. Quel en sera l'impact ? Communiquer vous permettra de prévenir les principaux intéressés mais aussi de disposer d'informations supplémentaires qui peuvent s'avérer indispensables.

Une fois tout le monde prévenu, n'oubliez pas de vérifier que personne d'autre n'est connecté à la machine. Si c'est le cas, prévenez les du reboot grâce à la commande `wall` :

```
# who
root      pts/0      2010-08-04 10:24 (192.168.250.163)
bob       pts/1      2010-08-04 10:43 (192.168.250.152)
ralf      pts/2      2010-08-04  8:57 (192.168.250.178)
# wall
Reboot dans 5 minutes !
^D
```

Petite astuce : il est possible de communiquer directement avec un utilisateur grâce à la commande `write`. Par exemple, pour communiquer directement avec Bob :

```
# write bob
Salut bob !
Impossible de killer le process 'bidule' (bloque en D), je dois rebooter la machine
^D
```

Ce qui produira sur la console de Bob le résultat suivant :

```
bob@snafu$
Message from root@snafu on pts/1 at 10:27 ...
Salut bob !
Impossible de killer le process 'bidule' (bloque en D), je dois rebooter la machine
```

EOF

Enfin redémarrez le système :

```
# reboot
```

Lors du *reboot*, vous serez déconnecté de la machine (jusque là, c'est plutôt normal). Mais, avez-vous pensé à ce qui va se passer si la machine ne parvient pas à redémarrer et qu'elle demeure injoignable ? Avant de rebooter le système, prenez donc vos précautions en essayant d'anticiper au maximum ce qui peut arriver.

6 Diagnostiquer un problème réseau

Nous avons jusque ici abordé un ensemble de problèmes courants pouvant impacter un système Unix (manque de mémoire, charge trop importante, disques pleins, etc.). Une autre série de problèmes, qui n'est d'ailleurs pas particulière aux systèmes Unix, concerne la connectivité et la possibilité de joindre ou non certaines machines du réseau.

Le présent chapitre requiert un minimum de connaissances. Il faut notamment savoir ce qu'est une adresse IP, le protocole TCP ou UDP et un port réseau.

6.1 Tester un service TCP

La plupart du temps, un problème réseau est remonté à l'administrateur système par une phrase du type : "Il y a un gros problème ! La page web du serveur truc ne s'affiche plus". Cela manque un peu de précision ! Heureusement, des outils assez simples vont nous permettre d'établir un diagnostic plus précis.

Utiliser un outil dédié au service

La première chose à faire est de tester le service en tant que tel. Pour commencer, on peut utiliser un outil dédié au type de service impacté : un navigateur pour tester un service web, un MUA pour envoyer ou recevoir un mail, etc. Leur utilisation permet de se faire une première idée du bon fonctionnement du service.

Dans un deuxième temps, il est souvent nécessaire d'utiliser des outils de plus bas niveau afin d'établir un diagnostic plus précis. Par exemple, si l'utilisation d'un navigateur permet de vérifier le bon affichage d'une page web, des outils en ligne de commande, comme `telnet`, permettent d'établir des statistiques du type "le serveur HTTP fonctionne seulement 1 fois sur 2" ou de vérifier le bon déroulement protocolaire quand la communication est établie.

La commande `telnet`

A l'origine, la commande `telnet` était utilisée pour se connecter et accéder à la console d'un serveur distant. Cette commande n'est désormais plus utilisée

dans ce sens car le protocole sous-jacent pose un problème important de sécurité : les identifiants transitent en clair sur le réseau. Cela signifie que n'importe quelle autre machine ayant accès au même réseau et pourvue d'un *sniffeur* peut récupérer le login et le password d'une personne qui se connecte via **telnet**. Cette commande permet toutefois de tester la connectivité d'un service TCP. Par exemple, pour tester le service HTTP d'un serveur :

```
$ telnet fnord 80
Trying 192.168.0.42...
Connected to fnord.
Escape character is '^['.
^]
```

Dans l'exemple ci-dessus, suite à la commande **telnet**, le message **Connected to** montre que la machine distante a accepté la connection. Une fois cette communication établie, il est possible d'entrer du texte pour tester plus en profondeur le fonctionnement du service. Pour sortir, il faut taper **']'** (Ctrl + crochet fermant).

Voici un autre exemple :

```
$ telnet fnord 25
Trying 192.168.0.42...
telnet: Unable to connect to remote host: Connection refused
```

La réponse **Connection refused** de la part du serveur indique que l'on est parvenu à contacter la machine distante, mais que l'accès au service (ici le service 25, qui correspond au *smtp*) à été refusé. Les raisons les plus courantes sont soit que le service n'est pas lancé ou qu'il est tombé, auquel cas il faut le relancer, soit que la machine est protégée par un *firewall* qui interdit l'accès à ce service.

L'exemple ci-dessous illustre une autre situation courante nettement plus ennuyeuse :

```
$ telnet eris 80
Trying 192.168.0.23...
^C
$
```

Dans ce cas, le serveur *eris* n'a pas répondu et il a fallu interrompre la commande (avec **^C**) pour reprendre la main. Cela semble indiquer un problème de connectivité plus sérieux et il faut pousser plus en avant notre investigation. Mais que faire ?

Une bonne méthode de résolution est de partir des couches les plus "hautes" et de descendre petit à petit vers les couches applicatives et protocolaires plus basses et plus complexes. Après l'échec du test du service TCP du serveur, l'étape suivante va donc être de tester la connectivité IP de celui-ci.

6.2 Tester la connection IP vers une machine distante

L'outil universel pour tester la connection IP vers une machine est la commande **ping**.

La commande ping

Cette commande envoie des requêtes ICMP de type *echo request* à la machine distante. A leur réception, celle-ci doit en principe répondre par un paquet ICMP de type *echo reply* (un *pong*). La présence de cet échange permet de valider au moins en partie la bonne connection entre les deux machines en présence. Par exemple, pour tester la connection vers la machine *fnord* :

```
$ ping fnord
PING fnord (192.168.0.42) 56(84) bytes of data.
64 bytes from fnord (192.168.0.42): icmp_seq=1 ttl=59 time=102 ms
64 bytes from fnord (192.168.0.42): icmp_seq=2 ttl=59 time=138 ms
64 bytes from fnord (192.168.0.42): icmp_seq=3 ttl=59 time=173 ms
64 bytes from fnord (192.168.0.42): icmp_seq=4 ttl=59 time=103 ms
^C
--- fnord ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 102.513/129.432/173.305/29.183 ms
```

Ce teste apporte plusieurs informations intéressantes. On observe que le serveur distant a répondu à l'ensemble des requêtes avec des temps s'étalant de 102 à 173 ms. Ces faits sont résumés sur les deux dernières lignes du listing. Notez aussi que la taille des trames ICMP envoyées, par défaut de 64 octets, est ajustable. Cela peut par exemple permettre de tester la MTU d'un lien.

Plusieurs raisons peuvent expliquer l'échec de la commande **ping**. Mais avant de conclure à un problème de connectivité, il est faut exclure les problèmes annexes liés à la résolution de nom en testant directement l'IP du système distant :

```
$ ping 192.168.0.42
```

Si le test ci-dessus échoue, alors oui, il y a certainement un problème de connectivité. Mais attention! Parfois, le **ping** échoue tout simplement parceque le système distant est protégé par un firewall qui filtre les requêtes ICMP. Si en revanche ce test réussit alors qu'il échouait quand il prenait directement en argument le nom du serveur, il faut s'orienter vers un problème de résolution de nom.

La commande traceroute

Cette commande n'est pas présente sur tous les systèmes, elle est pourtant bien pratique! Lors d'un test de connectivité, elle permet de suivre le cheminement des paquets de données jusqu'à la machine distante. En cas de problème, elle permet donc de voir l'élément fautif de la chaîne. Il est préférable d'utiliser l'option **-n** qui désactive la résolution de nom :

```
$ traceroute -n 192.168.0.42
```

6.3 Tester la résolution de nom

Pour comprendre cette section, il est nécessaire de comprendre les principes de la résolution de noms et le rôle des *DNS* au niveau d'Internet. Juste pour

résumer, un DNS est un service permettant d'établir une correspondance entre une adresse IP et un nom d'hôte.

La commande `host`

Obtenir une IP à partir d'un nom d'hôte est très simple avec la commande `host` :

```
$ host fnord
fnord has address 192.168.0.42
```

En cas d'échec de la résolution, l'option `-v` permet de connaître l'adresse du serveur de nom interrogé. L'adresse de ce serveur est configurée dans le fichier `/etc/resolv.conf` :

```
$ cat /etc/resolv.conf
domain kallisti.org
nameserver 192.168.0.2
```

Il est possible de tester directement un serveur de nom en spécifiant son adresse dans la requête :

```
$ host fnord dns2.kallisti.org
Using domain server:
Name: 192.168.0.3
Address: 192.168.0.3#53
Aliases:
fnord has address 192.168.0.42
```

6.4 Connaître son adresse IP et analyser la configuration réseau

Si absolument aucune machine n'est joignable avec la commande `ping`, alors peut-être le problème vient-il de votre machine? Cette section décrit les commandes permettant de retrouver la configuration réseau de la machine sur laquelle vous êtes.

La commande `ifconfig`

La commande `ifconfig` permet notamment de visualiser la configuration des différentes interfaces réseaux. Par extension, elle permet de connaître les IP configurées :

```
$ ifconfig -a
```

La commande `route`

Quand on peut joindre uniquement les machines de son subnet sans pouvoir en sortir, le problème est sans doute lié à un défaut de routage. La commande `route` permet d'analyser la table de routage :

```
$ route -n
```

La commande iptables

Des règles locales de *firewall* peuvent également être incriminées dans des problèmes d'accès à des machines distantes. La commande **iptables** permet de lister les différentes règles de filtrage :

```
# iptables -L -n
```